Computer Communications 195 (2022) 424–440

Contents lists available at ScienceDirect

Computer Communications

journal homepage: www.elsevier.com/locate/comcom

A machine learning-based optimization for end-to-end latency in TSN networks

Daniel Bezerra [a,*], Assis T. de Oliveira Filho [a], Iago Richard Rodrigues [a], Marrone Dantas [a], Gibson Barbosa [a], Ricardo Souza [b], Judith Kelner [a], Djamel Sadok [a]

[a] Grupo de Pesquisa em Redes e Telecomunicações, Universidade Federal de Pernambuco, Avenida Prof. Moraes Rego S/N, Recife, Pernambuco, Brazil
[b] Ericsson Research, Rod. Eng. Ermênio de Oliveira Penteado, 57 - 5, Indaiatuba, São Paulo, Brazil

ARTICLE INFO

Keywords:
TSN
End-to-end latency
Network optimization
ULL

ABSTRACT

Ultra-low latency (ULL) is a challenging but nonetheless essential network requirement to achieve in many contexts. Industrial automation (IEC/IEEE 60802), in-vehicle communications (IEEE P802.1DG), audio and video bridging (IEEE Std 802.1BA), aerospace, and 5G fronthaul (IEEE 802.1CM) applications demand a low network latency in the order of few milliseconds or even microseconds. ULL services target a more controlled latency for end-to-end device communication and, in critical environments, near real-time connections. Time-Sensitive Networks (TSN) aim at providing deterministic connectivity over IEEE 802 networks guaranteeing packet transport with bounded latency, low packet delay variation, and low packet loss. This work analyzes TSN latency mechanisms and examines how they influence E2E latency. Moreover, it builds a regression-based surrogate model that enables the creation of optimization models to find optimal configurations for a given TSN network. Our findings span the discussion of the characteristics of each individual TSN latency mechanism to the benefits achieved by the proposed optimization models and using these to achieve optimal E2E latency.

1. Introduction

End-to-end (E2E) network latency represents the time it takes for packet traversal between two end devices to complete. Its control is both challenging and essential as it significantly influences network applications' behavior and, consequently, their performance. Traditional local area network technologies, such as the IEEE 802.3 Ethernet standard, reduce E2E latency to the order of tens of milliseconds. Despite this, some applications require lower operational latency, often referred to as Ultra-low Latency (ULL). They include modern applications such as augmented/virtual reality, cyber–physical systems, vehicular communications, aerospace, and real-time control applications. For instance, according to [1], modern industrial applications should have an end-to-end latency on the order of a few microseconds or a few milliseconds.

ULL allows a more controlled latency for end-to-end devices and, in critical environments, near real-time connections. Moreover, in a heterogeneous environment, where there typically are different levels and forms of communication interfaces, reliable means to guarantee coverage of ULL are ideal for managing different types of network traffic.

In 2012, the IEEE 802.1 Time-Sensitive Network Task Group (TSN TG) was founded to standardize the real-time and critical Ethernet dependencies. The task group is still active in the standardizing process for time-sensitive capabilities and constantly contributes to defining and refining a wide range of 802.1 sub-standards. These cover aspects that include clock synchronization, frame preemption, redundancy management, and scheduled traffic enhancements [2]. Overall, the TSN TG set as its mission to provide deterministic connectivity through IEEE 802 networks, i.e., guaranteed packet transport with bounded latency, low packet delay variation, and low packet loss [3].

Network optimization comprises technologies and techniques that contribute to improving network performance. Latency optimization is challenging as it requires careful design and is often much more challenging to achieve than, for example, improving bandwidth performance and lowering packet loss. Network engineers constantly develop and invest in new higher bandwidth communication technologies that improve bandwidth and lower packet loss. On the other hand, Delay results from complex and diverse sources such as packet processing, queuing, transmission, and propagation delays. The latter component, propagation delay, cannot be controlled due to physics' limitations on signal propagation time in nature. Emerging ULL services attempt to

* Corresponding author.
E-mail addresses: daniel.bezerra@gprt.ufpe.br (D. Bezerra), assis.tiago@gprt.ufpe.br (Assis T. de Oliveira Filho), iago.silva@gprt.ufpe.br (I.R. Rodrigues), marrone.dantas@gprt.ufpe.br (M. Dantas), gibson.nunes@gprt.ufpe.br (G. Barbosa), ricardo.s.souza@ericsson.com (R. Souza), jk@gprt.ufpe.br (J. Kelner), jamel@gprt.ufpe.br (D. Sadok).

https://doi.org/10.1016/j.comcom.2022.09.011
Received 17 May 2022; Received in revised form 27 July 2022; Accepted 8 September 2022
Available online 15 September 2022

deal with some of these challenges. One can even enlist the help of emerging machine learning (ML) techniques. These have recently made considerable breakthroughs in diverse areas, e.g., speech recognition and computer vision. ML tries to construct algorithms and models that learn to take decisions directly from data without following predefined rules [4]. They fall within the class of data-based models that allow a black-box type of system analysis as they do not require previous domain expertise, hence their attractiveness. In addition, ML techniques are known to adapt and learn from new situations. When applied in the context of this work, they should learn about network behavior, extract its main features and create representative models that can be used to predict its performance. The main goal behind our study is to develop a solution whereby a network engineer, using the proposed model, may predict latency (model output) for a given network configuration setup (module input parameters). The adopted technique, namely regression, predicts an output that can be used next to search for optimized network latency values.

Before delving into the network's modeling, one needs to visit and analyze the primary latency control technologies adopted as part of the TSN standards. In order to obtain insights into the adopted TSN mechanisms across a broad range of configurations and network setups, a simulator is used. Latency information for different network configurations, including frame preemption, Credit-Based Shaper (CBS), and Gate Control List (GCL), is observed and collected as the primary metric for this study. The collected data from the simulations is then fed into a regression technique. This second phase aims to learn and ultimately find the optimal set of mechanisms that minimize network latency.

The findings of this work involve TSN analysis regarding several latency control mechanisms and the role they play in the control of end-to-end latency. Some of the findings of this work include:

1. Although preemption aids a critical flow by stalling the smaller-priority flows, the general E2E latency may increase despite preemption in some cases;
2. A correctly adjusted GCL significantly impacts network latency while cooperating in parallel with other latency mechanisms;
3. The CBS credit algorithm impacts CBS-based flows and the best-effort ones. Then, a balance among the credit logic needs to be established and applied in the network;
4. Choosing the proper CBS parameters is essential as this determines how long a flow needs to recover its credits. A longer credit recovery time allows other lower-level priority flows to steal its resources from higher priority CBS flows.

Moreover, we develop optimization models that recommend optimal network configurations. The work experiments with the evolutionary algorithms NSGA-II and SMPSO. Both suggest that the three latency mechanisms can work together and sum up their contributions towards lowering latency; however, some adjustments regarding frame size and ideal idleSlope CBS parameter values are needed to reach minimal E2E latency.

The remainder of this paper is organized as follows. Section 2 describes the latency control mechanisms designed by the respective standards. Section 3 reviews related research regarding latency optimization in TSN networks and current general latency optimization methods. Then, the scenarios and setup used in this work are described in Section 4. This section also details the optimization process and how we developed our surrogate model. Moreover, Section 5 presents the initial scenario results and the initial findings regarding TSN latency mechanisms. Subsequently, Section 6 details the expanded scenario results having the initial Scenario as a basis for a more complex setup. Then, with a more complex and near-reality scenario, the optimization results are described in Section 7, detailing the optimization models results and network optimization. Then, after the model's results, we detail how they would be applied in a real network scenario in Section 8. Finally, our final remarks are detailed in Section 9.

2. TSN overview

According to [5], QoS guarantees for time-sensitive applications are important, given the synchronization demands and time-constraints that are required. Therefore, it becomes a preeminent requirement to provide a flexible QoS infrastructure that allows dynamic the configuration of the QoS parameters. A robust QoS support is tolerant to faulty conditions, meaning that the overall system should not be affected in the presence of abnormal activities. As a result, TSN provides QoS guarantees by supporting different standardized features with specific objectives. The deterministic demand for network latency generates the development of new mechanisms that guarantee low bounded delay and deterministic packet scheduling. To this end, the TSN standards introduce traffic control technologies that perform:

- Frame Preemption — IEEE Std 802.1 Qbu-2016 [6]
- Credit-based Shaper (CBS) — IEEE Std 802.1 Qav-2009 [7]
- Gate Control List (GCL) — IEEE Std 802.1 Qbv-2015 [8]
- Path Control and Reservation — IEEE Std 802.1 Qca-2015 [9]
- Timing and Synchronization for Time-Sensitive Applications in Bridged Local Area Networks — IEEE Std 802.1AS-2011 [10]

This work mainly focuses on latency optimization. As a result, it is concerned with three TSN mechanisms selected to study their impact on latency and understand the interplay among them. According to [11], frame preemption, CBS, and GCL are the three chief mechanisms that guarantee latency control in a TSN network while following the network determinism. However, their combined usage can be optimized to reach even higher levels of network performance by minimizing the end-to-end latency.

These three latency technologies are detailed in the following sections but first, a brief discussion about the Asynchronous Traffic Shaping (ATS) standard [12], last published in 2020 for latency control, is made. ATS aims to achieve bounded low transmission latency for mixed-type traffic without global time synchronization [13]. Although it fits in the low-latency goal, ATS can be understood as a complementary mechanism or even a counterpart for the GCL since ATS works with asynchronous flows. Some works [13,14] state that GCL, when adequately set with accurate and precise gating schedules, generally achieves the specified latency bounds for both sporadic and periodic traffic. In contrast, GCL performs relatively well for sporadic traffic; but struggles with high loads of periodic traffic. Therefore, we discarded from the present study the ATS mechanism and focused on latency mechanisms that work well under synchronous networks. In addition, the ATS implementation in the network simulator used in this work and described in further sections is not available. Therefore, the ATS impact on E2E latency analysis is considered for future work.

2.1. Credit-based shaper

IEEE 802.1 Audio Video Bridging (Ethernet-AVB) refers to a set of technical standards that specify mechanisms developed to ensure controlled latency communication across Ethernet networks. Among the defined standards, the IEEE 802.1 Qav [7] specifies the queuing and forwarding rules for Ethernet-AVB switches. It has been designed to reduce buffering in switches and hosts. These rules are supported by a Credit-based Shaper (CBS) mechanism. A few years later, TSN also integrated the IEEE 802.1 Qav as a basis for its work. A CBS relies on several parameters for the control of traffic. The work in [15] offers the following parameter definitions:

- **IdleSlope:** Determines the rate at which a flow accumulates credits. It takes place while frames are still waiting for transmission. Frames are only transmitted when the credit value is equal to or greater than zero. In other words, the IdleSlope plays a central role in determining the bandwidth reserved for a flow. A higher value means a shorter time it takes a flow to acquire credits

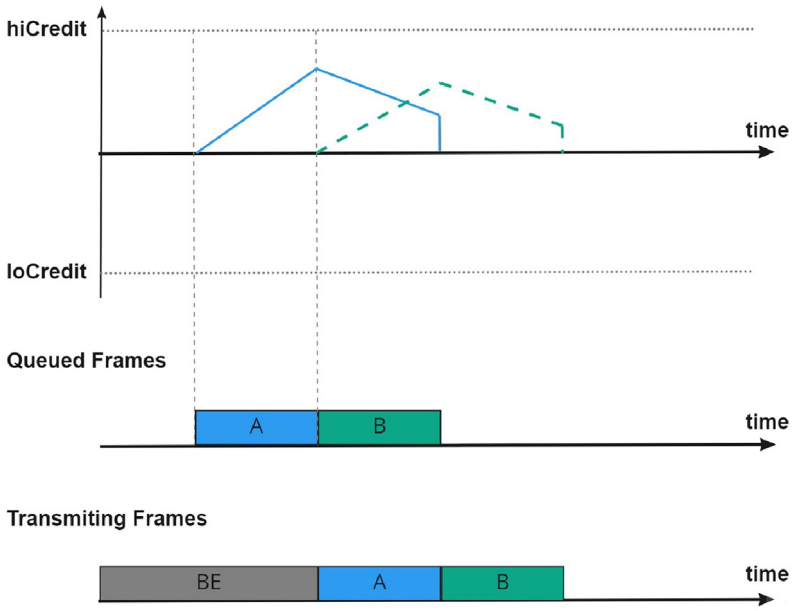


Fig. 1. Classical CBS model. Figure based from [16].

for sending new frames and hence makes it more competitive and aggressive. When the credit is negative, and there are no transmitted frames, credit increases at an idleSlope rate until it reaches zero value.

- **SendSlope:** Represents the rate at which credit is reduced while frames are being transmitted. It is set as $sendSlope = idleSlope - port\_transmission\_rate$.
- **hicredit:** Defines the maximum of credits that can be accumulated. $hicredit = max\_interference\_size * (idleslope/port\_transmit\_rate)$. It is important to set a limit for the amount of acquired credits, and when the bucket is full, no more credits may be obtained. As a result, no flow may acquire large amounts of credits, allowing it to monopolize egress bandwidth.
- **locredit:** Defines the minimum amount of credits that can be reached. $locredit = max\_frame\_size * (sendslope/port\_transmit\_rate)$

Both Ethernet-AVB and TSN implement priority queuing scheduling. Unlike strict priority queuing, a high-priority CBS traffic class cannot starve lower priority traffic classes. This is ensured by the use of the above two parameters: *idleSlope* and *sendSlope*. CBS is associated with two or more priority classes where frames are transmitted following a credit-based rule. A two-classes CBS process is illustrated in Fig. 1. In the example, frames from classes A and B arrive at the switch port for forwarding. Meanwhile, the TSN switch must also transfer non-time dependant flows; such flows are categorized at a Best-effort class (BE class). The transmission of time-dependent flows is only allowed when: 1. The corresponding class credit is bigger or equal to zero; 2. There is no higher priority class frame in the sending queue, or a higher priority class credit does not have enough credits to start transferring; 3. No other frame is being sent (including lower priority frames).

Under CBS, a frame is dequeued and transmitted at a *sendSlope* rate, while the credit increases at an *idleSlope* rate and when a higher priority frame is not being transferred. According to Fig. 1, when the single Frame-A arrives, it is queued due to the presence of best effort (BE) frames already being transmitted by the switch. An *idleSlope* rate increases the credits while the current transmission is taking place. After finishing the ongoing frame transmissions, the Frame-A credits decrease by a rate of *sendSlope*, and the switch initiates the transmission of Frame-A. Meanwhile, Frame-B arrives during frame-A transmission. Then Frame-B has to wait while its credit accumulates. When Frame-A ends transmitting, the credits are set to zero. Now, frame-B has enough credit to be sent forward. After its transmission, the credit is set to zero, and no more frames are in the queue.

2.2. Frame preemption

The IEEE 802.1 Qbu standard [6] introduces a preemption mechanism to the TSN standards specification. A switch egress port may preempt a transmitting frame; in other words, it may stop its transmission, usually due to a higher priority frame. Port egress queues are classified into Express and preemptable queues. As the name implies, an express queue transmits frames in a non-stop manner, whereas a preemptable queue may have its frame transmission paused and resumed later.

To put it more simply, the standard allows pausing a frame transmission in preemptable queues, transmitting a frame from an express queue, and then resuming the other frames from the paused state. Fig. 2 depicts a preemption scenario. Three frames from three different queues are transmitted through an egress port. The Scheduled Traffic (ST) flow is set to an express queue while Flow A and B are preemptable, i.e., they can be paused and broken. Flow B is ready to transmit at time 0; since the credit is zero, it can be transmitted continuously. At time 4, an ST frame arrives in the queue and requires instant transmission. As a result, the Flow B transmission and the credit count stop to allow the express queue to send any frames it enqueued. All credits remain constant (paused state) during ST frame transmission until time 8. Then, in this example, Flow B resumes the transmission of its interrupted or preempted frame. Frame B continues to transmit until the end of the frame, and the flow is finished. Then, flow A starts transmitting and decreasing the corresponding credit. If another ST frame arrives, the same process happens again.

Although preemption is a beneficial mechanism used mainly for protecting scheduled traffic, there are situations where it is not applied. For example, it does not make sense to interrupt the transmission of an almost entirely transmitted frame. The problem is that, in practical terms, it is almost impossible for a switch to know the number of bytes that remain to be transmitted, especially when using cut-through switches. This is mainly because the time it takes to preempt a frame may become more extensive than the actual transmission of the remaining bytes. Following a similar thought, it is not worth preempting tiny frames. Therefore, the TSN set of standards stipulates that frames with a size less than or equal to 123 Bytes cannot be preempted.

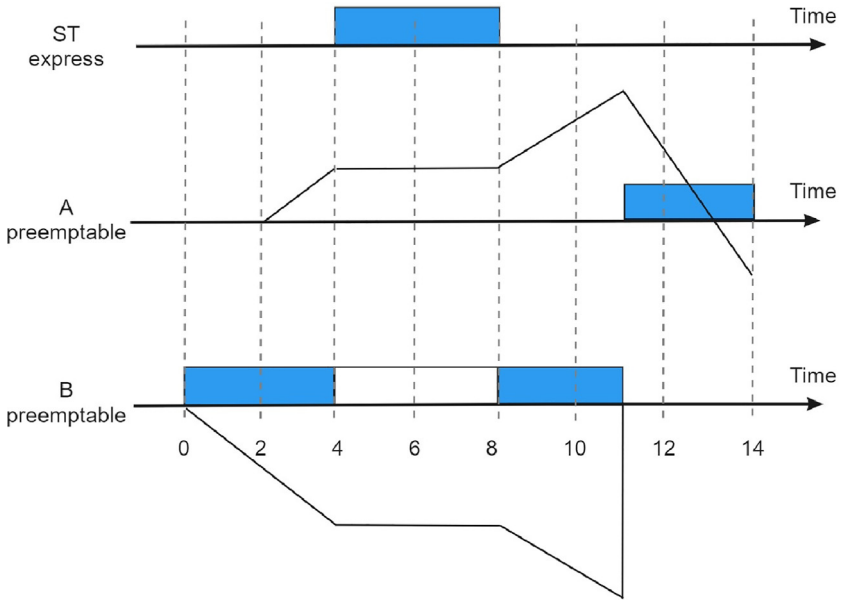


Fig. 2. Frame Preemption occurs in preemptable queues to allow the transmitting of frames queued on express queues.

2.3. Gate control list

As TSN focuses on time-sensitive flows, a time-aware shaper (TAS) implements a control mechanism to handle such flows more efficiently. The IEEE 802.1 Qbv standard [8] adds a Gate Control List (GCL) to manage the sending queues and select which will send the packets forward. In other words, each queue is associated with a gate that opens/closes to allow the frame transmission according to a predefined schedule (Fig. 3). Therefore, this established schedule determines when a frame is dequeued and sent through the egress port.

As depicted in Fig. 3, different queues receive three types of flows. The ST queue handles only the Scheduled Traffic. The Pn queues manage the different priority flows where n varies from 1 to 8. Pn is the *n*th priority queue in the switch. There is also a queue for Best-effort traffic. GCL operation affects the traffic and the real-time properties the network guarantees. If all queues allow the transmission during a time interval, the queue priority defines which one forwards the data; therefore, a strict scheduling policy is ideal to prioritize different data types (e.g., time-sensitive, AVB, and best-effort data). The gates opening and closing schedule follows a time-triggered (TT) paradigm, i.e., the scheduled event overrides the queue priority allowing or not the frame transmission to the egress port.

The TT mechanism on the GCL allows two types of traffic (scheduled and non-scheduled). Scheduled traffic is defined as having network requirements, e.g., maximum end-to-end latency and minimal jitter. Moreover, scheduled traffic is considered to be high-critical and periodic [2]. Scheduled traffic is related to scheduled queues to ensure latency and jitter requirements. Meanwhile, non-scheduled traffic uses priority queues that do not interfere with scheduled traffic. Ideally, low-priority queues are used for non-scheduled traffic.

3. Related work

The TSN set of standards initiated development as early as 2012 and evolved since then with published standards such as [6,18]. The latency technologies described in Section 2 are well-defined mechanisms, and many works discuss them, as shown next.

Preemption, as defined in TSN, follows the following two basic setup steps: (i) Assign priority to each flow; (ii) Assign each flow to a preemption egress port. Moreover, it is beneficial for critical data since it reduces the delay of time-critical frames occasioned by competing non-critical traffic. However, preemption directly affects other flows during transmission. According to [19], preemption performance degrades when the number of express frames is high. When too many flows are essential, this is close to having none. Traditional Quality of Service (QoS) mechanisms, such as the class-based Internet Differentiated Services (DiffServ) have always cautioned network traffic engineers when creating high priority DiffServ Express Traffic (EF) and advised to keep this well below the 5% level. However, even when maintained below an acceptable threshold, TSN express traffic can be delayed by other express traffic. However, when the volume of express traffic exceeds the stated threshold, the effect of express traffic on each other causes prejudice in latency requirements. To circumvent and improve the preemption method, some works [20,21] propose novel preemption mechanisms in the MAC layer to obtain a multi-level preemption in which higher priority preemptable class may preempt lower priority preemptable class.

Due to the poor predictability of traffic, a low priority frame, such as BE frame, may extend into a cycle dedicated to a time-aware scheduled frame and delay its transmission leading to latency violations. TSN introduces guard bands to mitigate this. They allow the suppression of any transmission for the duration of a maximum Ethernet frame size. On the other hand, their use leads to a waste of bandwidth resources. To avoid this, a switch could inspect if a current low priority frame is small enough to be transmitted without extending into a time-aware scheduled frame or that it must wait for the next opportunity. This is not easy to implement in a switch. As already stated, preemption is often preferred to make room for the transmission of higher priority traffic as it does waste resources, as in the case of using guard bands.

Nonetheless, preemption also may cause overhead when other flows are interrupted by the guardband; therefore, [22] propose a mechanism to calculate and reduce the guardband size and, consequently, the overhead. As preemption is an important research topic, many works have studied timing analysis from frame preemption in TSN networks alongside other traffic shaping mechanisms and scheduled traffic [23,24].

Typically, preemption alone is insufficient to guarantee real-time communication, and additional TSN mechanisms, such as CBS, are required. The work in [25] describes a TSN scenario for an industrial automation system. The authors use network calculus to provide a bounded latency for TSN. They combine CBS and Strict Priority as QoS-based shaping mechanisms to evaluate frame delay. Although network

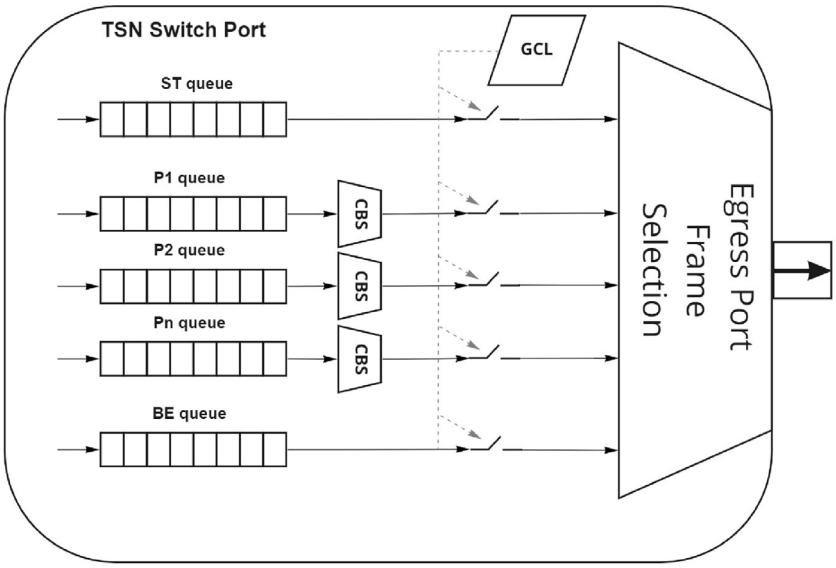


Fig. 3. Classical GCL model [17].

calculus helps evaluate upper end-to-end delay bounds, its use is limited to simple network topologies. The integration of GCL and CBS is also investigated by [26] aiming to analyze a joint CBS and TAS scheduling model in an AVB context. It describes how it changes transmission frames' relative order, leading to bursts and worst-case scenarios for low-priority streams. Regarding latency analysis, [27,28] conducted a study about the worst-case latency analysis of different shapers in automotive and avionics contexts.

The works related to the latency control mechanisms typically analyze the worst scenario to provide upper bounds and propose better algorithms or compare how they work in different circumstances. Overall, latency improvement remains a current research topic approached by diverse efforts. The authors in [29] aim at using TSN in an industrial system to guarantee strict QoS and enable deterministic real-time applications. One considered QoS requirement is latency, as demanding applications need periodic and critical event-based messages. They build a TSN-enabled industrial scenario and use Thing Description (TD) vocabulary to generate QoS capabilities and classify the "things" in the network.

Meanwhile, [30] proposes an algorithm based on the greedy randomized adaptive search procedure (GRASP) to schedule both AVB and Time-Triggered traffic. Given an existing partial scheduling solution, several feasible schedules exist for a flow. The achieved schedule can then be post-processed to minimize end-to-end latency.

Instead of improving latency, some works target the optimization of TSN networks based on scheduling or traffic, using latency as a requirement stated as part of a general network optimization problem. The authors in [31] have examined the scheduling of real-time traffic, whereby the transmission schedules are computed through optimization methods(Satisfiability Modulo Theories (SMT) and Optimization Modulo Theories (OMT)). The optimization constraints for problem formulation are based on general TSN configurations in terms of the characteristics of Ethernet frames, physical links, frame transmissions, and end-to-end requirements. The optimization problem is modeled to compute schedules to achieve low latency and bounded jitter during transmission while considering a comprehensive set of parameters. Other works adopt machine learning techniques to find the optimal configuration of TSN networks [32–34]. Although they use latency as a learning constraint, they do not specifically optimize the end-to-end latency. Instead, they focus on optimizing scheduling and transmission.

End-to-end latency optimization remains an actual concern while developing real-time networks. TSN is currently evolving and under standardization. Hence it opens up many avenues for future research, especially considering ULL requirements. Therefore, this work aims to evaluate the impact of the developed low-latency mechanisms and understand the interplay among them. Moreover, the availability of an intelligent configuration integrated control strategy will likely be critical for providing effective ULL to end-users of a given network.

4. Experimental setup

This section lists the adopted metrics, describes the setups, and presents the used experimental scenarios. This work considers two scenarios designed with different objectives despite relying on the same TSN underlying mechanisms. Section 4.1 describes all potential metrics investigated in this work and justifies the reason behind their analysis. After, Section 4.2 defines the tools used to build the scenarios and run the planned experiments. Moreover, Sections 4.3 and 4.4 detail both scenarios. Section 4.3 discusses some preliminary studies and offers some initial insights regarding how the different network mechanisms and features affect how TSN flows interact with each other and the network itself. Then, the scenario is expanded and detailed in Section 4.4. A more in-depth analysis requires such a new expanded scenario. Machine learning is applied next to extract more generalized and well-founded conclusions regarding TSN network provisioning and latency optimization.

4.1. Performance parameters

The different parameters used in the setup of the conducted experiments fall into two groups: Network parameters and TSN parameters. Network parameters represent typical network configuration features that can affect the flows and influence network performance. On the other hand, the described TSN parameters are related to the setup of the different TSN mechanisms as part of the network configuration. These are listed below.

- Network parameters
  - Bandwidth: Indicates the maximum amount of data transferred in a given time. Since this work focuses on data transmission, it is a significant parameter to consider to analyze how it affects the overall end-to-end latency. TSN controls the egress data and, therefore, the bandwidth allocated to flows, using its traffic shaping and latency mechanisms.

Table 1
All evaluation parameters and the respective values for both network scenarios.

	Values
Bandwidth	{100 Mbps; 1 Gbps; 10 Gbps}
Frame size	{500B; 1000B; 1500B}
TS algorithm	{Strict Priority; CBS}
Idle slope	{0.1; 0.2; 0.3; ...; 0.8; 0.9}
Gate control	{True; False}
Preemption	{True; False}

  - Frame Size: There are two crucial frame attributes relevant to the present study, namely, Frame size and Maximum Transmission Unit (MTU) size. The MTU specifies the maximum number of bytes that can be encapsulated in an Ethernet frame, which is 1518B (or 1522B when it accommodates a four bytes VLAN tag). A frame consists of an Ethernet header (14 Bytes), the payload (1500 Bytes), and the Frame Check Sequence (4 Bytes). Given that the primary role of a TSN switch is frame forwarding, it is relevant to experiment with different frame sizes and evaluate how they affect TSN latency.

- TSN parameters
  - Traffic Shaping Algorithm (Strict Priority vs. CBS): Traffic shaping algorithms allow different traffic flows on the same network to coexist. They redistribute ingress frames giving traffic a smooth and regular shape with no bumps or bursts. The CBS, presented in Section 2.1, schedules each flow transmission based on a credit rule and its priority. Unlike CBS, Strict Priority scheduling only considers priority when selecting the next frame from the egress queues to transmit on a port. Frames from higher priority queues are transmitted first. Given their differences, this work looks at the impact of using schedulers such as strict priority and CBS.
  - IdleSlope: This is a crucial resource-related parameter. It determines the credit rate that CBS uses to allocate to a flow. The idleSlope rate can vary depending on the supported port speed. There is a need to carefully engineer this parameter among the competing flows to achieve latency requirements for all flows in a TSN network.
  - GCL: As described in Section 2.3, the GCL handles the flows based on a scheduling mechanism. Then, enabling and disabling the use of GCL in a switch could lead to different end-to-end latency results.
  - Frame Preemption: The possibility of preempting a frame to favor the transmission of a higher-priority flow is needed to ensure high-level priority flows meet their deadlines. However, preempted frames may accumulate at the egress queue, causing additional delays. There is also the actual overhead this mechanism introduces.

To summarize, the parameters used in the experiments, the values, and ranges they assume while running the experiment are detailed in Table 1. These parameters need to be optimally configured to optimize end-to-end latency.

4.2. General experiment setup

The experiments run in an Ubuntu 20.04 LTS machine with 16 GB RAM memory and Intel Core i7-6700 CPU. Both scenarios, described in the next section, are implemented in the OMNeT++ simulator [35] with NeSTiNg, a well-known open-source TSN plugin [36]. NeSTiNg inserts TSN features into the standard network functions. It offers the main TSN mechanisms: scheduled TSN traffic, priority marked frames and CBS.

In both scenarios, a dumbbell topology (Fig. 4) creates a bottleneck among switches and force flows to compete for bandwidth. The machines connected to Switch 1 are named Hosts, while those connected to Switch 2 are named Sink. The number of machines may vary between 1 to N, according to the different objectives of the tested scenario. The flows are sent from the Host to the Sink, where the Nth Host sends traffic to the Nth Sink. The sending rate of each flow is calculated in order to occupy the network's maximum capacity and is detailed in the following sections.

Moreover, the three TSN mechanisms seek to provide a better service experience targeting latency control guarantees. Each mechanism operates to improve different aspects of the network and, altogether, improve the overall E2E latency. According to [11], it is expected that such mechanisms individually provide the following guarantees: 1. Preemption fully guarantees the transmission delay of the highest priority data frame; 2. CBS provides offers to allocate bandwidth of different streams; 3. GCL guarantees that all streams are strictly transmitted according to a current list. In this work, a goal is set within the experiments to achieve an optimal E2E latency given the guarantees provided by each mechanism individually.

A key TSN feature is detailed in IEEE 802.1ASrev [37] which defines the network time synchronization protocol. TSN provides a global network clock to synchronize all network hardware and guarantee well-determined communication. It also offers a fault-tolerance system for reelecting a new clock master (also known as Grandmaster) in case of failure. The simulator establishes time synchronization globally, through an event queue, as if a Grandmaster was coordinating the switches. The flows are marked, in the senders, with priority tags using VLAN identifiers (VID) in their VLAN-tagged frames. The flow priority can range between 0 to 7 in a 3-bit vector named priority code point (PCP), defined at [38].

The parameters described in Section 4.1 influence each traffic flow individually. The assumed values are averaged per flow using a configurable time interval. We consider the following two scenarios:

- Experimental Scenario: Defines an initial version of a scenario for TSN overall analysis. Some of the parameters detailed previously can be analyzed individually to test their impact in a TSN network.
- Extended Scenario: Specifies a more complex scenario with different types of flows. It is a more realistic scenario used to create the surrogate model.

4.3. Experimental scenario description

Aiming to learn which parameters affect the TSN network and are worth studying in the second expanded scenario, an experimental scenario is first proposed. It allows an understanding of the behavior of some network characteristics in a TSN environment and obtains insights into how they impact the end-to-end latency. Some of the findings are coherent with state-of-the-art presented in Section 3. As depicted in Fig. 5, the hosts named AV (Audio-Video), MP (Medium Priority), and BE (Best-effort) are responsible for sending traffic to the opposite side of the network through Switch1. AV, MP, and BE hosts mark their frames with priority codes (PCPs) 7, 4, and 0, respectively. The Three hosts are connected to Switch1, and the other hosts (Sink 1 to 3) are connected to Switch2. The sending rates of all the flows are calculated to occupy all available bandwidth. We set the scheduled AV flow with a 354B frame size to be sent each 400 µs. This time, GCL was not used in this experiment. MP and BE flows compete for the available bandwidth and have the same configuration where 1500B packets are sent at a rate of 200 µs. This scenario runs under an available network bandwidth of 100 Mbps.

CBS is the main traffic shaper algorithm we evaluate in this work; however, in an attempt to understand the interplay among flows and CBS operation, part of the experiments use other schedulers.

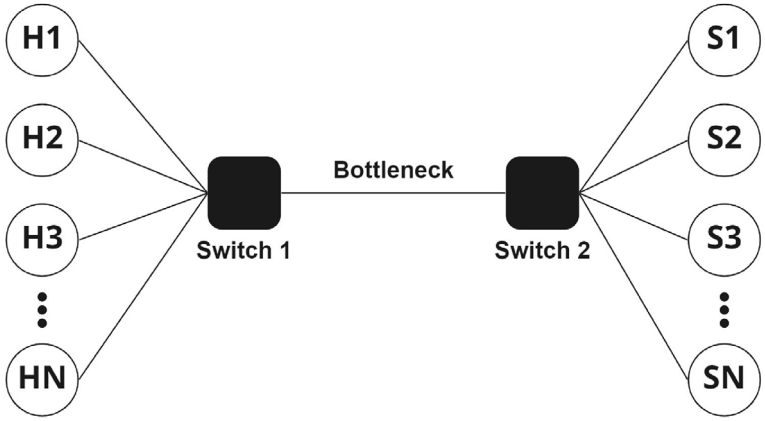


Fig. 4. Dumbbell topology for both scenarios.

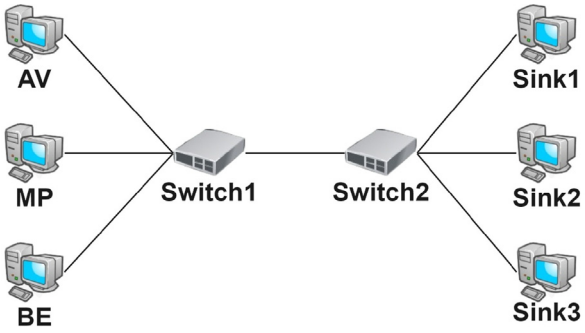


Fig. 5. Initial network topology for TSN analysis.

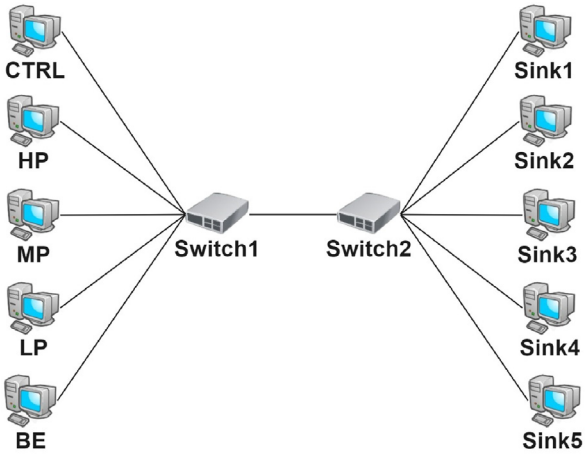


Fig. 6. Dumbbell network topology.

4.4. Expanded scenario description

Next, an extended scenario is built to examine the impact of the three TSN mechanisms working together and understand how they influence each other. The following topology is the main scenario on which the additional experiments are based. As depicted in Fig. 6, a dumbbell topology is used to set the network topology. It has ten hosts and two TSN switches. Five Hosts are connected to Switch1, and the other Hosts (Sink 1 to 5) are connected to Switch2. The CTRL host is responsible for sending the scheduled traffic. It plays the exact same role as that of the AV host mentioned in the previous scenario. The hosts, HP (High-priority), MP (Medium-priority), and LP (Low-priority), are accountable for the time-sensitive traffic. It is essential to mention that only the time-sensitive traffic uses CBS; therefore, it has an idleSlope applied to the forwarding mechanism. The BE host is the Best-Effort traffic representing the usual Ethernet traffic that is not time-critical. Table 2 shows the priority level of each host and the frame size of all flows used during the experiments. CTRL and BE frame sizes are fixed, while the CBS flows use two frame sizes, a smaller value of 500B and a higher one of 1500B. It is essential to conduct latency evaluations for flows with different frame sizes as these typically change in an entire deployed network. A dumbbell topology induces a bottleneck across the central link between switch1 and switch2. The sending rates of all flows are calculated to occupy all available bandwidth together. We set the scheduled CTRL flow to use a 354B frame size sent each 400 µs, at the begging. HP, MP, and LP flows constantly compete for the available bandwidth with an individual frame sending rate of 110 µs. This scenario uses an available network bandwidth of 100 Mbps.

The expanded scenario is used to run the experiments and generate the output for the regression models. The experiments combine different configuration parameters, as listed in Table 4. In addition to the varying frame size, preemption may be enabled or disabled, while idleSlope takes the values shown in Table 4. The GCL may also be either enabled or disabled, resulting in the following behavior. When disabled, the GCL does not affect any flows, and no gates open or close. However, when enabling GCL, the gates are designed to change their status periodically. The queues periodically assume the configuration set in Table 3. In our experiment, the CTRL flow sends data periodically at the beginning of each 400 µs interval, which is the time window that the GCL prioritizes scheduled traffic, along with Best-effort traffic. Then, the GCL changes the configuration to prioritize CBS-only traffic. Then, all gates related to non-scheduled flows are opened. This GCL configuration repeats each 400 µs. The 8-bit vector in the "GCL configuration" column represents each queue status for all ports for the switch, where bit 1 represents the fact that traffic is allowed to be transmitted and bit 0 indicates that the frames are put on hold. Each bit represents a PCP queue ranging from PCP 7 (highest PCP) to PCP 0 (lowest PCP), i.e., in a GCL configuration of 10000011 means that only queues 7, 1, and 0 allow transmission while all others are currently blocked.

Table 2
Flow priorities and frame sizes.

	Priority	Frame size
CTRL	7	354B
HP	6	500B; 1500B
MP	4	500B; 1500B
LP	2	500B; 1500B
BE	0	500B

Table 3
GCL configuration when enabled in the network.

Interval time	GCL configuration
0 μs–100 μs	10000001
100 μs–300 μs	01111110
300 μs–400 μs	01111111

Table 4
Network parameters and respective values used in the experiments.

	Values
Frame size	{500B; 1500B}
Idle slope per CBS-flow	{0.1; 0.3; 0.5; 0.7; 0.9}
Gate control	{True; False}
Preemption	{True; False}

Moreover, five end-to-end latency values are collected for each experiment. They correspond to the delay experienced by each of the five flows. As a result, there are 4000 executions due to factor combinations. The data related to a given experiment run, including its input parameters and observed end-to-end delay (output metric), are used to generate the regression models. Model input parameters used by the adopted machine learning algorithm include: 1. The preemption and GCL status (enabled/disabled); 2. The frame size for each CBS flow; 3. The corresponding idleSlope for each flow; 4. The percentage of dropped frames for each flow. Lastly, the five latency values are also included in the dataset, but these are used as targets (part of the output layer) for the machine learning algorithm.

4.5. Latency optimization

In Section 2, we selected three standardized technologies used for latency control. Our results show that depending on the characteristics and objectives of a network configuration, the use of a single or two mechanisms is often sufficient. First, a regression-based surrogate model captures the system's main characteristics and discovers relationships among the flows and the end-to-end latency. Then, the surrogate model is the basis for latency optimization.

A surrogate model is an engineering method used to calculate a result of interest that is not trivial to compute; Instead, an output model is used. It can work as an alternative for simulators, i.e., instead of running simulations for an extensive period, it can model the simulator's results and output the outcome almost instantly. As detailed in Fig. 7, a set of outputs achieved through simulations can be costly because of running time or dependence on computation resources. However, a surrogate model built from scratch with mathematical theory or machine learning training from fewer simulation runs can generate a near-perfect set of outputs. The training phase, depicted in Fig. 7, uses fewer simulation runs as input to generate the surrogate model. The training is based on machine learning and works as an understanding phase to generate simulation behavior models. After the training, the surrogate model can substitute the simulator and generate the corresponding output, according to what was learned.

This work uses PyCaret [39], an open-source machine learning library in Python that automates machine learning workflows. The training phase in this work is based on regression models that learn the network behavior composed of five flows in different network setups. Each E2E latency result represents a single flow from a respective host, influenced by a combination of the previously described parameters. PyCaret takes a set of parameters as input to build regression models. The library initiates the transformation pipeline and prepares the setup by taking two mandatory parameters: data and target. After selecting the inputs that are characterized as each of these two groups, PyCaret trains and evaluates the performance of all estimators available in the library using cross-validation. The evaluation generates a score for each estimator; then, according to pre-defined evaluation metrics (e.g., mean absolute error, mean squared error, R-squared), the best one is chosen to generate the model. Next, the generated model predicts a target value given a data set as input.

After learning, each regression model is used as part of the surrogate model to output the final end-to-end latency at the receiving machines. Therefore, the surrogate works as a replacement for the OMNeT++ simulator. The parameters are a set of network features and the three TSN mechanisms detailed in Section 2. To verify the accuracy of the regression model, pyCaret leverages a checking tool that compares real data to the output generated by a given model. When running the tool for each regression model composing the surrogate, the E2E latency values estimated by the models achieve a prediction accuracy of around 98.8% when compared with the simulated output data.

The output of the surrogate model is used as the base for the optimization model. The result of the simulations, which scenario was previously described in Section 4.4, are used as a training set for the surrogate model. A total of 77% of the data is used as a training set, and 23% is used as the test set. All latency results are normalized to understand and manipulate the end values quickly. After training, the model is used by the adopted optimization algorithms. The tool used to optimize the end-to-end latency is the jMetal [40], a framework for multi-objective optimization with metaheuristics. jMetal offers two evolutionary algorithm implementations for multi-objective problem-solving to evaluate the predictions generated through the regression models: the Non-dominated Sorting Genetic Algorithm II (NSGA-II) and the Speed-constrained Multi-objective Particle Swarm Optimization (SMPSO). Evolutionary optimization (EO) algorithms use a population-based approach in which more than one solution participates in an iteration and evolves to a new population in each iteration. The multi-objective approach provides a set of Pareto-optimal solutions which need further analysis to arrive at a single preferred solution. Such a solution results from a trade-off among the objectives analyzed by the EO algorithm.

In this work, the initial objective is to minimize the end-to-end latency. However, as detailed in further sections, optimizing one objective can illustrate an inefficient scenario, e.g., the model could reach an optimal E2E latency. However, there could be a flow that would suffer an almost 100% loss. Therefore, this work's scope requires a multi-objective analysis with latency and frame loss. Further details regarding the optimization and the overall TSN results are given in the following sections.

5. Experimental scenario results

Before discussing the surrogate model's results, we present those from the experimental scenario. As discussed, the experimental scenario serves as a preliminary scenario to study the impact of the used parameters and understand how they affect a TSN network. This section describes the results and discusses some of the parameters' impact on a TSN network.

5.1. Traffic shaping algorithms

CBS is described in Section 2.1. It is a traffic shaping (TS) algorithm that limits the switch's egress port transmission. However, a non-limited port uses the Strict Priority (SP) algorithm. An SP port sends frames strictly according to the flow PCP, i.e., there is no credit-based strategy. Each flow's TS algorithm is altered during the current tests to observe the impact of combining SP and CBS in the same scenario according to Table 5. There are eight runs in total, and each of them represents a different flow TS algorithm.

The first experimental results show that the CBS algorithm does not work well in some situations with SP. More specifically, runs 3, 4, 7, and 8 (where the MP flow uses SP) did not allow BE traffic to transmit any frames. As MP's priority is higher than BE's, and they compete to be transferred, BE never gets transmitted and is entirely

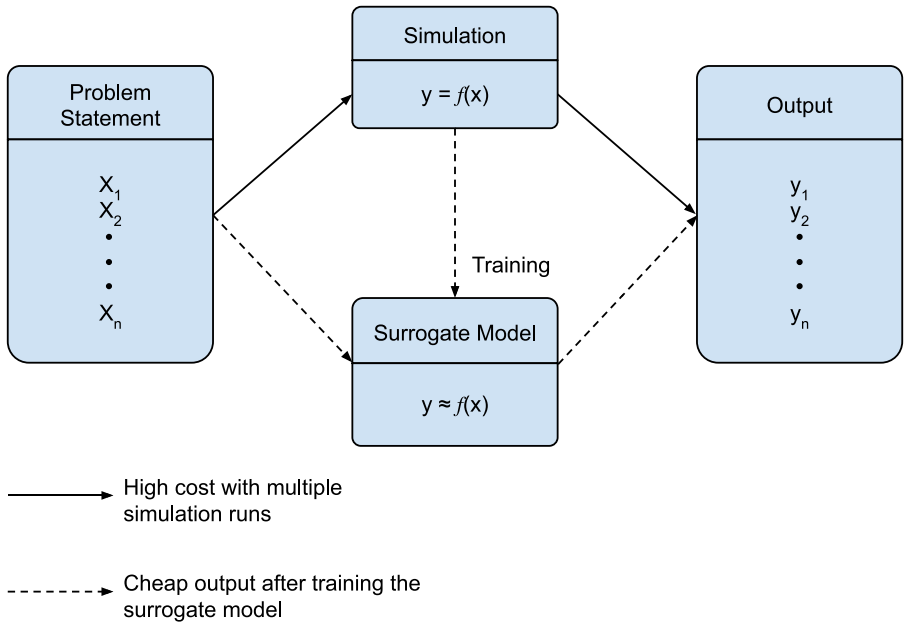
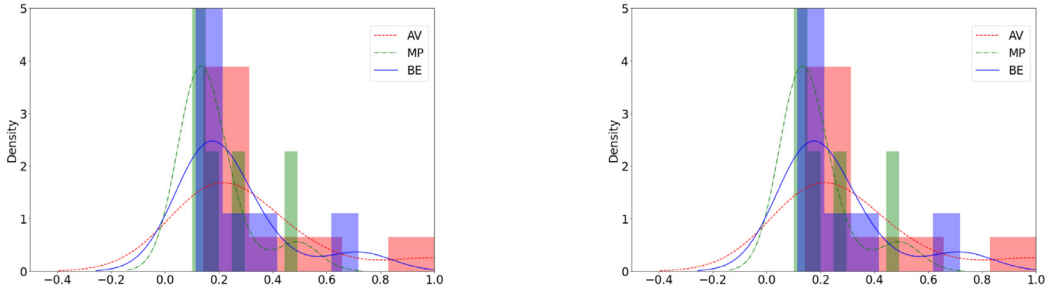


Fig. 7. Surrogate model construction compared with regular simulation.



(a) Frame latency distribution while all hosts are configured as CBS

(b) Frame latency distribution while only the highest PCP host is configured as SP

Fig. 8. Frame latency distribution.

Table 5
Flow priorities and frame sizes.

Run	Algorithm		
	AV	MP	BE
#1	CBS	CBS	CBS
#2	CBS	CBS	SP
#3	CBS	SP	CBS
#4	CBS	SP	SP
#5	SP	CBS	CBS
#6	SP	CBS	SP
#7	SP	SP	CBS
#8	SP	SP	SP

Table 6
Flow priorities and frame sizes.

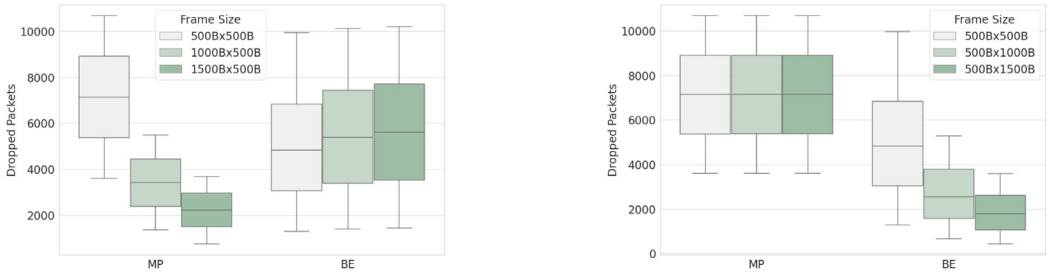
Frame size			
Group #1		Group #2	
MP	BE	MP	BE
500B	500B	500B	500B
1000B	500B	500B	1000B
1500B	500B	500B	1500B

discarded by the switch, whether using CBS or SP. Simply put, BE traffic suffered starvation. It occurs because Strict Priority does not pause frame transmission and keeps the egress port occupied—consequently, a higher-priority flow using SP blocks any lower-priority flow. In essence, this is why TSN introduces CBS, which is less aggressive than SP, where high priority flows may consume all the resources and starve the lower priority ones or increase their delay in accessing the network resources uncontrolled.

In addition, runs 1, 2, 5, and 6 present similar results among each other. More specifically, runs 1 and 5 present the same distribution result, just like runs 2 and 6. Fig. 8 depicts the distribution of runs 1 and 5 and shows that using CBS or SP for the AV flow maintains the same pattern. As the AV host is a scheduled flow, it does not compete with other flows and has the maximum priority; therefore, it is always transmitted by the switch.

5.1.1. Frame size impact on TSN

After analyzing the impact of the algorithms, the chosen configuration to continue observing and studying was set to all flows using CBS (run #1 from Table 5). As CBS uses a credit-based logic, it would be wise to verify the impact of the frame size in competing flows. MP and BE are the competing flows, and the frame size combination varies according to Table 6.



(a) The number of dropped frames when MP frames size is equal to or larger than BE frame size.

(b) The number of dropped frames when BE frames size is equal to or larger than ME frame size.

Fig. 9. Different frame sizes configuration and how it affects other flows at the same network.

For Group #1, the MP frame size is greater or equal to BE frame size, whereas, for Group #2, the MP frame size is less or equal to BE's. Frame sizes used in the experiment are 500B, 1000B, and 1500B. The results depicted in Fig. 9 show per-flow and the number of frames dropped at each configuration for both groups.

Fig. 9(a), represents Group #1. A higher priority flow (MP) with a larger frame size tends to lose fewer frames than a lower priority flow (BE). It occurs because CBS credit logic needs to transmit all frames before selecting the next frame in the queue. If the higher priority frame is oversized, the lower priority waits longer and gets inserted in the outer queue because the credit of the high-priority frame is negative; therefore, the number of dropped frames in BE raises as the MP frame size gets larger. Meanwhile, Fig. 9(b), representing Group #2, shows that a higher priority frame with a smaller size keeps a stable frame drop rate for MP itself. This configuration helps BE have a lower drop rate since the frame size is more significant, and the credit logic keeps transmitting BE flow while holding MP frames, even though MP has a higher priority. This compensation could be attractive depending on the objectives of the network manager, i.e., the manager can choose which flow he should focus on. One may partially sacrifice the higher-priority and transmit the lower-priority one or concentrate entirely on transmitting higher-priority flows while allowing some loss of regular traffic.

5.1.2. Bandwidth impact on TSN

The last batch of tests is concerned with the analysis bandwidth influence on TSN. The tests were run with 100 Mbps, 1 Gbps, and 10 Gbps link bandwidth values, always in a competing network environment. Fig. 10 depicts the overall end-to-end latency values for each idleSlope value in different bandwidths. The idleSlope values for the following tests are equally set for each flow. In the first experiment, the three idleSlope values are set at 0.1, then 0.2, and so on until an idleSlope of 0.9. Although the latency values for each flow differ among the three bandwidths, their behavior remains similar.

For BE host, latency always rises with the idleSlope increase. The latency for MP follows the same behavior in all bandwidth tests and decreases until the idleSlope value of 0.5; it presents some variations while idleSlope ranges from 0.5 to 0.7 and then continue to decline from 0.7 to 0.9. Meanwhile, the AV host also shows a latency decrease, but at bandwidth 10 Gbps and idleSlope 0.6 and 0.7, an anomaly is caused by held frames at the egress queue. A wait caused by the credit mechanism contains some frames causing the latency's mean value to increase. However, for all other idleSlope values, the declining behavior of the idleSlope is maintained.

An ANOVA test ensures that the similarities among the flows set with different bandwidth values are accurate. Grouping by Host, the ANOVA test verifies the similarity among flows in different bandwidths, i.e., three groups are composed of the same flow in different bandwidths (Group AV is composed of the three AV flows at 100 Mbps, 1 Gbps, and 10 Gbps). Using a significance level of 0.05, the ANOVA provides the values listed in Table 7. The high P-values from Group HP and BE indicate a high probability that the difference observed in the groups is arbitrary. Therefore, there is no significant difference among the groups. The *p*-value of Group AV is near 0; however, after analyzing the data, the impact is given by the waiting in the credit mechanism described previously. Therefore, the difference in the group is caused by CBS, not by bandwidth.

Table 7
P-value for each group of flows.

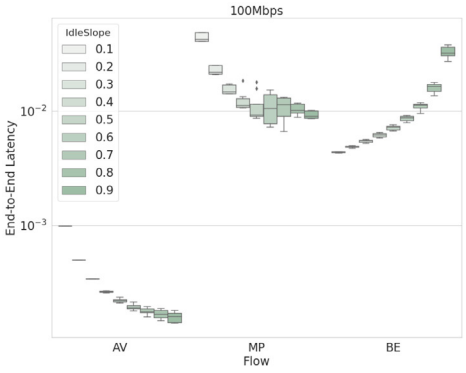
	p-value
Group AV	7.64e−6
Group MP	0.9202
Group BE	0.974

According to the described analysis, the bandwidth does not significantly affect the end-to-end latency. Therefore, the expanded scenario focuses on the 100 Mbps bandwidth to evaluate the overall TSN latency afterward. Further, idleSlope is still used to evaluate end-to-end latency. However, each flow has a different idleSlope value, implying that each flow has a credit value that will impact the latency and even other flows. This scenario will be investigated further ahead.

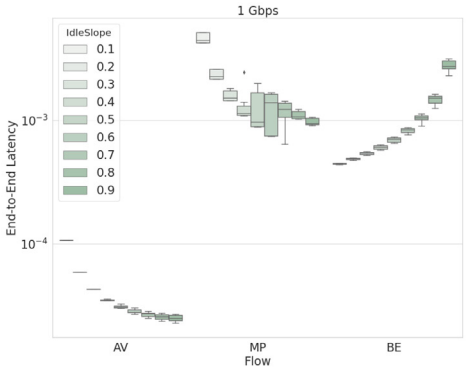
5.2. Overall analysis

In this section, some parameters were initially studied to ascertain how they impact TSN networks. The findings described previously are summarized in the following points.

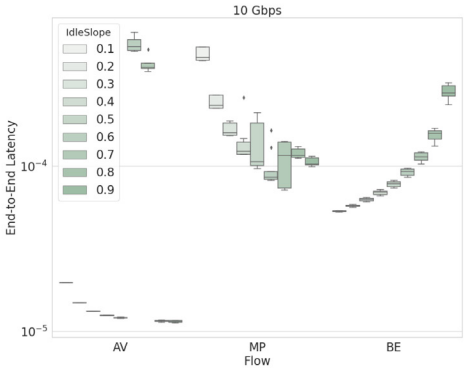
- Traffic Shaping:
  - High-priority flows with a Strict Priority algorithm tend to occupy the egress port, not allowing lower-priority flows to be transmitted, increasing their delay uncontrollably, and even starving these.
  - A high-priority scheduled traffic is not influenced by the algorithm it operates.
- Frame Size:
  - In a preemption-free environment, the low-PCP flow waits longer if a higher-priority frame has a larger frame size than lower-priority frames. Therefore, it could lead to dropped frames in the queue.
  - In a preemption-free environment, if a higher-priority frame has a smaller frame size than a lower-priority frame, the drop frame from a high-PCP flow keeps a stable rate. Meanwhile, the low-PCP flow decreases the drop rate when frame size increases.



(a) IdleSlope values for 100 Mbps



(b) IdleSlope values for 1 Gbps



(c) IdleSlope values for 10 Gbps

Fig. 10. IdleSlope variation for different bandwidth.

- Bandwidth:
  - The bandwidth impacts the TSN. Since it reflects the volume of the transmitted data, a higher bandwidth contributes to lower latency; however, statistically, the data delivery stays the same for all bandwidth values, i.e., there is a difference in the E2E latency in a 100 Mbps and a 10 Gbps but the amount of data delivered is statistically similar.

6. Expanded scenario analysis

After analyzing the experimental scenario, we restrict the set of parameters in this next experiment according to the previous results. Bandwidth is fixed at 100 Mbps, and the Strict Priority algorithm is discarded; only CBS is used as traffic shaping. All other parameters and the values they assume during the simulations are maintained. To clarify, the remaining parameters are Frame Size, IdleSlope, Gate Control List, and Preemption.

6.1. Frame preemption

As detailed in Section 2.2, Frame Preemption pauses frames transmission in preemptable queues and proceeds to transmit in express queues typically. As express queues are related to more critical flows, the expanded scenario sets the CTRL flow to use an express queue. A switch stalls the current transmitted flows and prioritizes the CTRL flow. Since it is scheduled traffic, this interruption occurs periodically.

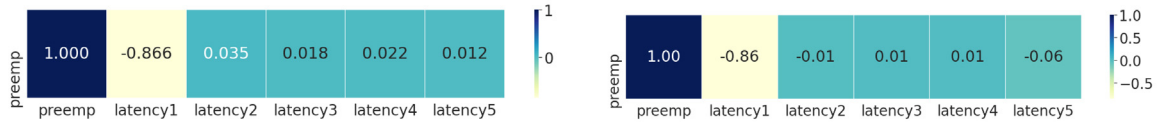
Fig. 11 represents the correlation between Frame Preemption and all the collected E2E latency metrics. Fig. 11(a) represents the scenario with GCL enabled while Fig. 11(b) represents the scenario with GCL disabled. At both heat maps, the preemption correlation with the impact in E2E latency from CTRL host is around −0.86. The correlation suggests that when preemption is enabled (value set to 1), the latency value of the CTRL flow significantly decreases and impacts directly in the end value. Moreover, the preemption does not affect the other flows significantly. However, there is a reduced impact when preemption is combined with the GCL. As depicted in Fig. 11(a), while the GCL mechanism is disabled, the correlations lead to near null values. When the GCL is enabled, all values drop (Fig. 11(b)), indicating that enabling GCL in a preemption-enabled environment leads to smaller values of E2E latency. Although the difference is small, it is still evident while activating the GCL.

Therefore, the usage of preemption is not always beneficial to the network. It greatly benefits the flow associated with the express queue while harming the other flows. Ideally, it should be activated depending on the network's objectives. Preemption could be enabled if the network usage is not affected by flow interruptions. Although the preemption results suggest the stated findings, preemption is still a good choice in the surrogate model and remains a possible solution to minimize latency.

6.2. Frame size

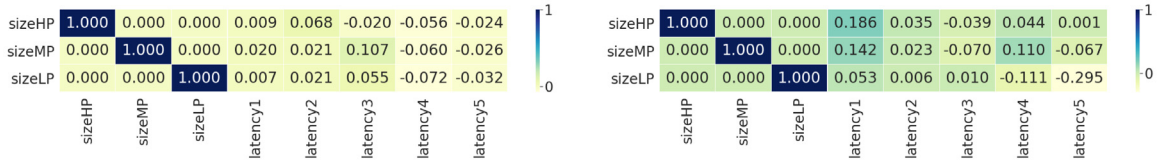
The overall correlation values are close when observing the frame size and latency values with gate control enabled or not; however, interesting behaviors can be seen individually. The frame size follows similar patterns from what was described in Section 5.1.1. It is worth mentioning that the following analysis does not involve preemption.

Fig. 12(a) depicts the correlation of each CBS-based flow with all latency values without using the GCL mechanism. Latency1 (CTRL flow) is lightly affected by different frame sizes as it represents the highest-PCP flow and is also a scheduled flow, leading to a minimum effect on its E2E latency. For CTRL, latency1 increases as the frame sizes also increase. On the other hand, latency5 (BE flow) is the lowest-PCP flow and depends on the credit mechanism to be sent, i.e., BE frames



(a) Preemption correlation with GCL disabled (b) Preemption correlation with GCL enabled

Fig. 11. Preemption correlation with all end-to-end latencies to evaluate how the mechanism affects each flow.



(a) Frame size correlation with GCL disabled (b) Frame size correlation with GCL enabled

Fig. 12. Frame size correlation with all end-to-end latencies to evaluate the impact of different frame sizes in each flow.

are transmitted when all the other flows have a positive credit and cannot transmit due, for instance, to lack of packets in their queues; Therefore, latency5 (BE) decreases while the CBS-flows' frame sizes increase. Although both flows are affected, the impact is minimal. However, a more noticeable impact can be seen in latency3. The most impacting frame size variation for latency3 is the size of MP frames. The bigger the size, the higher the latency. A larger MP frame size with a medium priority implies a higher latency value when competing at the egress port. The other correlation values are near zero, meaning a good correlation among frame sizes and latency values could not be found.

However, a strong correlation could be detected after enabling the GCL, as depicted in Fig. 12(b). The first noticeable change is the impact of the frame sizes on latency1. Flow HP (PCP 5) affects the E2E latency of flow CTRL with a 0.19 correlation, meaning that the bigger the size of HP frames, the bigger the CTRL latency, deteriorating the end-to-end latency of the highest priority flow. The same logic can be applied to the MP flow size (PCP 4), which has a 0.14 correlation. Meaning that GCL tends to deteriorate high PCP-flows if their frame size is larger. Meanwhile, latency5 (PCP 0) considerably decreases the correlation to −0.29 with the LP flow and −0.067 with MP flow, indicating that GCL helped decrease the E2E latency when the MP and LP flows have a larger frame size.

6.3. IdleSlope and CBS effect on latency

The idleSlope is the rate the credit mechanism uses to increase while waiting in the egress queue. Recall that the CBS flows are HP, MP, and LP. Fig. 13(a) depicts the correlation between latency and idleSlope values while the gate control is disabled. The CBS flows do not affect the CTRL end-to-end latency because the PCP value guarantees that CTRL flow is transmitted after a lower-priority frame is dequeued. On the other hand, all three CBS flows directly affect the BE flow as expected. As the idleSlope values for the CBS-enabled flows increase, the BE latency tends to decrease. The correlation with each flow is around −0.25, meaning higher idleSlope values allow a smaller latency5. Comparing the two flows with gate control enabled (Fig. 13(b)), the CTRL flow maintains the same behavior, but BE behaves differently. As the idleSlope values from the three CBS-enabled flows increase, the latency of BE flow also increases, with a correlation value around 0.36 for the three flows, i.e., higher idleSlope values increase the BE flow's latency significantly.

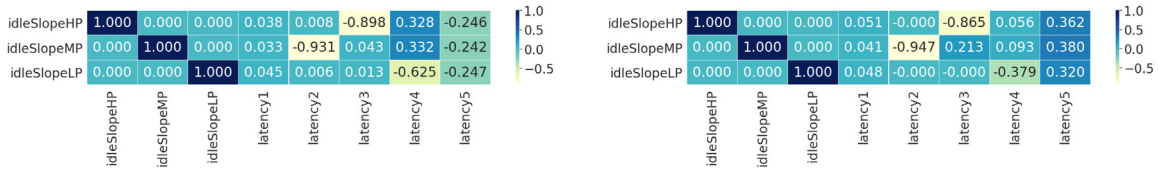
Among the three CBS flows, the idleSlope values directly correlate and affect each other. In both heat maps at Fig. 13, the MP idleSlope affects the HP latency indicating that a higher rate gives a more extensive time window to other high-PCP flows sent through the egress port, justifying the correlation around −0.95. The same logic applies to HP idleSlope affecting the MP latency, with a correlation value around −0.9. Notably, a higher MP idleSlope negatively affects itself. At the same time, gate control is enabled, with a correlation of 0.21, (Fig. 13(b)) because the TSN network prioritizes the high-priority flows; Therefore, a higher MP idleSlope greatly benefits the HP flow while prejudices the MP flow itself.

The LP idleSlope exhibit two different behaviors for each GCL scenario. While gate control is enabled (Fig. 13(b)) the LP latency is affected solely by the LP idleSlope. As LP idleSlope gets higher, the latency decreases; therefore, a higher idleSlope helps the LP traffic. However, while GCL is disabled (Fig. 13(a)), although the same logic applies to LP flow itself, it starts to negatively affect the other two flows(correlation of 0.33), mainly because the three flows are competing for a share of the switch capacity for transmission. Therefore, a higher idleSlope value for the lowest-PCP flow among the CBS-based flows tends to increase the other latency values.

6.4. Overall analysis

An analysis of the remaining parameters and their impact on TSN networks was conducted in this section. The findings described previously are summarized in the following points.

- Frame Preemption:
  - The findings regarding preemption suggest that using frame preemption helps the high-priority flows associated with express queues; However, at the cost of lightly harming E2E latency values experienced by other flows. Depending on how Frame preemption is set up in the network, it would benefit the critical flows but could harm the non-critical flows. Hence the level of express traffic must be maintained at a controllable level.
- Frame Size:
  - In a preemption-free scenario, a flow's frame size impacts E2E latency. Larger frame size for a higher-PCP flow could compromise its latency and other flows depending on how the egress queue or GCL holds the frames. Although there is an impact, the obtained results were inconclusive.
- IdleSlope and CBS:
  - The idleSlope variation during credit algorithm in the CBS flows impacts not only HP, MP, and LP flows but also the BE flow. The ideal idleSlope values should be found to avoid harming other flows; Therefore, a balance among idleSlope needs to be applied in the network.



(a) IdleSlope correlation with GCL disabled

(b) IdleSlope correlation with GCL enabled

Fig. 13. IdleSlope correlation with all end-to-end latency values to evaluate the impact in the network.

Table 8
R^2 values for each flow. The following values and models are calculated and analyzed by pyCaret.

Flows	R2 values	Chosen model
CTRL flow	0.9432	Extra tree
HP flow	0.9999	Decision tree
MP flow	0.9999	Decision tree
LP flow	0.9999	Decision tree
Best-effort flow	0.9979	Extra tree

- GCL:
  - The GCL significantly impacts the scenario. While working with preemption, GCL did not influence the results but did in the other experiments. During frame size analysis, it significantly changed the correlation values, indicating that should be a change in frame size depending on flow priority.
  - A high-PCP flow should have a lower frame size to achieve lower latency, while a low-PCP flow should have a larger frame size.
  - For idleSlope, the GCL significantly impacts the lower-PCP flows. As the BE flow is scheduled apart from the CBS-based flows, it has a different behavior that could benefit high-priority flows.
  - Meanwhile, while GCL was enabled, LP flow presented a more balanced latency result as the time window for sending frames is scheduled exclusive for the CBS-based flows, benefiting all three flows, including LP.

7. Optimization results

The expanded scenario contains three different sets of traffic: Scheduled traffic(CTRL), Time-sensitive traffic (HP, MP, LP), and Best-effort traffic (BE). As detailed in Section 4.5, the regression models learn the behavior of each flow and are used to predict each latency accordingly. The PyCaret library interprets the data and determines the ideal model for each latency. It trains and evaluates the performance of all estimators available in the library using cross-validation. It outputs the top-performing model based on the coefficient of determination (R^2). The best model follows an extra trees regressor for CTRL and BE, while the three CBS-based flows follow a decision tree regressor. The R^2 values for each regression model is listed in Table 8

Each model has feature importance as variables that most influence learning and prediction. The main points that can be extracted from each regression model are the following:

- The Preemption and GCL are the two features that influence the CTRL flow the most. As the CTRL host sends the scheduled flow, the activation of preemption and the gate switch tends to benefit the flow that follows the time synchronization.
- The three CBS-based flows are most affected by the dropped frames during transmission. As they compete to be sent by the egress queue, a high drop rate can harm the flows
- BE traffic is influenced by the dropped frames and the idleSlope values of the CBS-based flows. As the high-priority flows compete, there are fewer opportunities for best-effort traffic to be sent; if there is a high drop rate of CBS flows, the best-effort can move freely. Also, depending on the idleSlope values, the waiting time for the credit rate can benefit the lower-priority traffic.

Table 9
NSGA-II optimization results.

	Optimal value
Preemption	Off
Frame size HP	1500B
Frame size MP	500B
Frame size LP	500B
IdleSlope HP	0.1
IdleSlope MP	0.1
IdleSlope LP	0.9
Gate control	On
Frame drop HP (%)	99%
Frame drop MP (%)	0.23%
Frame drop LP (%)	0.28%
General End-to-End Latency	1.1294

The five models that map the behavior of all flows are used to build the surrogate model and predict the end-to-end latency. jMetal offers two genetic algorithms to compute the formulated optimization problem while using the surrogate model as the solution-finding process. Both NSGA-II and SMPSO implement the surrogate model to compare what each algorithm suggests as optimal. The surrogate model estimations of end-to-end latency are used to find the optimal values for the network when combining all flows.

According to the NSGA-II results (Table 9), frame preemption should not be used to reach an optimal latency since it degrades the overall latency, except the one for the CTRL flow. Ideally, the optimal size for the HP flow should be 1500B, while the other CBS flows should use a smaller frame size of 500B. The suggested IdleSlope values are low for HP and MP, as they directly influence each other and offer a higher value for LP since it is less significant than the others. Remember that a low IdleSlope implies a high SendSlope. Therefore, the HP and MP spend more time transmitting at the egress port. Moreover, the gate should be used to leverage the efficiency of the network. The GCL combined with CBS allows better network usage when correctly configured.

However, this optimal configuration tends to increase HP frame drops. As shown in Table 9, 99% of frames were lost. It makes the objective of minimizing latency not the only one to observe. To circumvent this issue, the surrogate model also minimizes the frame loss to avoid an optimal but not reasonable situation. The problem now focuses on two main objectives for both EO algorithms: minimize end-to-end latency and minimize frame drops. After including this demand, the model solves the new problem with both EO algorithms and generates two Pareto fronts of minimal latency and frame loss. The Pareto approximation (Fig. 14) depicts the results for NSGA-II. From the data, the solution that supplies the minimization demands is listed in Table 10. The main differences between the optimal result in Table 10 and the single-objective optimization presented in Table 9 are the usage of preemption and GCL and the idleSlope balance among CBS-flows. With GCL, the three CBS flows compete exclusively among themselves,

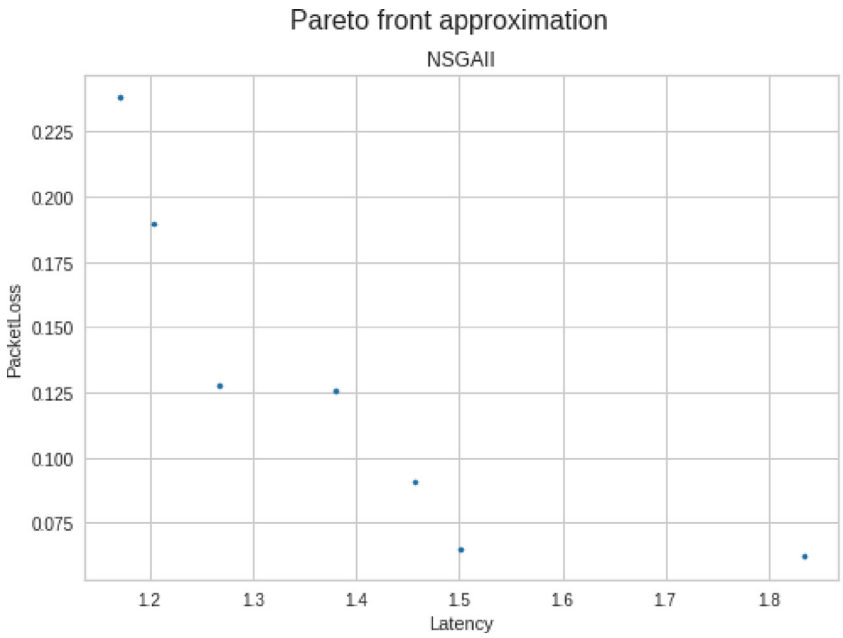


Fig. 14. Pareto front plot, for NSGA-II, for analyzing the best outcome for the multi objective solution.

Table 10
Multi-objective NSGA-II and SMPSO optimization results.

	Optimal value	
	NSGA-II	SMPSO
Preemption	On	On
Frame size HP	1500B	1000B
Frame size MP	1500B	500B
Frame size LP	1500B	1000B
IdleSlope HP	0.6	0.3
IdleSlope MP	0.2	0.2
IdleSlope LP	0.8	0.8
Gate control	On	On
Frame drop HP (%)	9.84%	1.08%
Frame drop MP (%)	8.23%	3.33%
Frame drop LP (%)	1.32%	0.98%
General End-to-End Latency	1.5014	1.3143

Table 11
Configuration for each parameter in the TSN network creation for each run, aiming to find a viable configuration set for the TSN network.

	Parameters values			
	Run #1	Run #2	Run #3	Run #4
Preemption	Off	On	On	On
Frame size HP	1500B	1500B	1500B	1500B
Frame size MP	1500B	1500B	1500B	1500B
Frame size LP	1500B	1500B	1500B	1500B
IdleSlope HP	0.9	0.9	0.5	0.4
IdleSlope MP	0.9	0.9	0.5	0.5
IdleSlope LP	0.9	0.9	0.5	0.7
Gate control	Off	On	On	On

leaving a free egress port for CRTL and BE, reducing the frame drop of some flows. Moreover, intermediate idleSlope values among the flows create a fairer environment while competing for the egress port.

Table 10 shows an optimal idleSlope value of 0.2 for flow MP. Although it is a low value, according to the heatmap and correlation analysis detailed in Section 6.3, a low idleSlope for MP, while GCL is enabled, lowers both the MP E2E latency and the HP E2E latency. This behavior can be analyzed in Fig. 13(b). Meanwhile, higher idleSlope values for the other flows (HP and LP) tend to decrease their respective E2E latencies. Moreover, as discussed in Section 5, the preemption tends to favor the CTRL flow while the CBS-based flows dispute with each other; therefore, the final latency value tends to be a little higher. Therefore, the General End-to-End Latency in Table 10 (1.5014) is higher than the end value presented in Table 9 (1.1294) because the model avoids a high frame drop and activates preemption to reach an optimum value.

Meanwhile, the SMPSO algorithm analysis provides a similar result to NSGA-II. The Pareto approximation (Fig. 15) depicts the results for SMPSO. From the data, the solution that supplies the minimization demands is listed in Table 10. The SMPSO chose to keep GCL and preemption enabled. Moreover, the idleSlope values are the same for LP and MP (0.8 and 0.2, respectively). For HP, the value considerably drops to 0.3. Also, compared to what was suggested by the NSGA-II model, the frame sizes for HP and LP are down to 1000B, and MP decreased to 500B.

Regarding the idleSlope values, the same logic detailed for the values selected by the NSGA-II model applies to SMPSO. According to Section 6.3, a smaller idleSlope for HP flow and MP flow implies in increasing the E2E latency for BE flow (PCP 0). However, there is a significant decrease in HP and MP latency. To compensate, the model lowers the frame size for all CBS flows. According to the heatmap at Fig. 12(b), a smaller frame size leads to a smaller E2E latency for HP, MP, and even BE. This compensation helped the General End-to-End Latency to achieve a smaller value. Also, there were fewer frame drops for all CBS-flows using the suggested configuration by the SMPSO model.

8. Applying the models in the network

This section details a preliminary use of the optimization models to show how optimal latency can be obtained in our TSN network. Since the optimization model is still not attached to the network, the process is still manual, but we plan to integrate it in future works, as detailed in the next section.

Initially, a network manager would physically build a network and then apply some control settings. We assume, in this example, that the network topology follows the extended scenario used in this work (6) and that the overall bandwidth is 100 Mbps. An initial configuration could be set up according to Run #1 in Table 11. In this configuration,

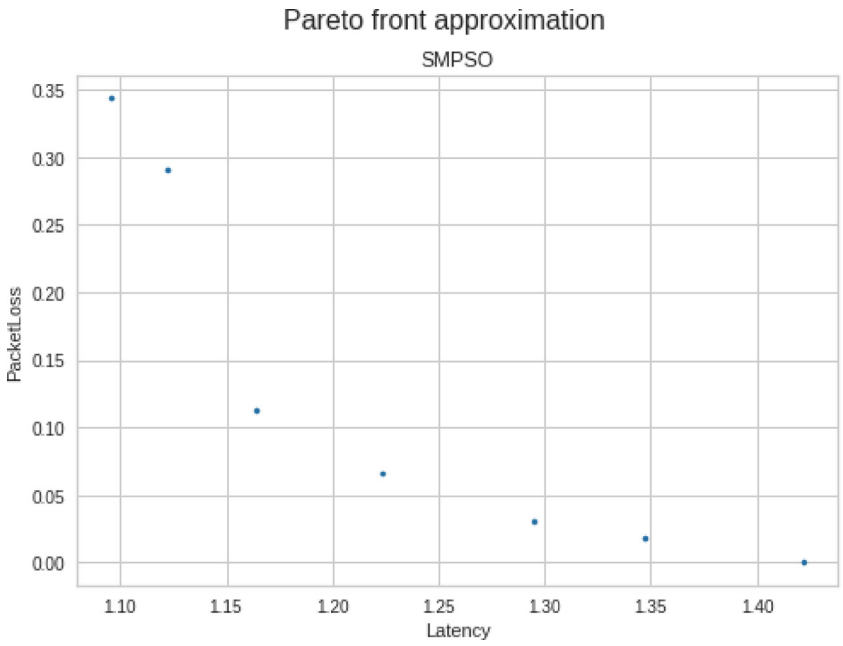


Fig. 15. Pareto front plot, for SMPSO, for analyzing the best outcome for the multi objective solution.

Table 12
E2E latency values per flow for each run.

	Latency (s)			
	Run #1	Run #2	Run #3	Run #4
CTRL	$1.6182E^{-4}$	$1.0287E^{-4}$	$1.0287E^{-4}$	$1.0133E^{-4}$
HP	0.0220	0.0277	0.0414	0.0485
MP	0.1119	*None*	0.0468	0.0414
LP	*None*	*None*	*None*	0.1242
BE	*None*	0.0349	0.0349	0.0464

Table 13
E2E latency values per flow for each optimized model.

	Latency (s)	
	Optimized-SMPSO	Optimized-NSGA-II
CTRL	$1.0030E^{-4}$	$1.0133E^{-4}$
HP	0.0531	0.03632
MP	0.0780	0.0783
LP	0.0261	0.1085
BE	0.1164	0.0464

all idleSlope values are assigned the value 0.9, and we adopt a frame size of 1500B. In addition, both preemption and GCL mechanisms are initially disabled. As the manager would not know the optimal setup, the first configuration would result in the E2E latency values listed in Run #1 shown in Table 12. Note that some E2E latency values present a value marked as *None*, meaning that the frames could not be transmitted because other flows blocked them; therefore, the setup should be adjusted.

During Run #2, the manager may enable the preemption and GCL in an attempt to reach a more balanced network since two more latency mechanisms should help the overall latency. Nonetheless, this new setup still results in blocked frames (Table 12), especially in the MP flow that was already being transmitted. Then, At Run #3, the new adopted configuration follows Table 11. It adjusts the idleSlope values to 0.5 for all the CBS flows. Observe that this helps the high-PCP CBS flows, but LP still suffers and experiences problems when sent by the queue. At Run #4 different idleSlope values are set, generating E2E latency values listed by Table 12 Run #4. All flows are transmitted and reach their destination.

Although the network can be set up according to the steps previously detailed, the optimization models can suggest an optimized setup that balances the flows, reaching an optimal E2E latency for the presented network topology. Table 13 details the E2E latency for each flow according to the optimal set up described in Section 7. Note that each model benefits from different flows. NSGA-II lowers the E2E latency for HP and LP while sacrificing MP. SMPSO decreases the scheduled traffic (CTRL) latency and greatly decreases that of LP. Overall, we conclude that using the optimized models guarantees an optimized and balanced network.

9. Conclusion

This work investigates the end-to-end latency optimization problem in a TSN network and proposes a regression-based surrogate model that delivers an optimal end-to-end latency. At first, the three main latency mechanisms were described and examined separately, and some insights provided a broader vision of how network traffic behaves under the TSN network. The three mechanisms complement each other. As explained in Section 5, frame preemption benefits the highest PCP flow exclusively, but when used with GCL, it can provide some advantages depending on the main objective of the network manager. Moreover, the combination of GCL and CBS tends to be the correct usage as flow frame size is known beforehand. Combined, they can balance the highest and lowest PCP traffic with the intermediate traffic (CBS flows).

The proposed optimization model uses a regression-based surrogate model to predict the best network features and plan optimal latency. The surrogate model detailed in this work uses the simulated data to find a collection of features that tends to minimize the end-to-end latency. The findings of this model show that it is essential to balance packet loss with optimal latency since the theoretical results of the three latency mechanisms (preemption, GCL, and CBS) can overlap. Ignoring packet loss as an essential second metric may degrade the overall outcome. The proposed surrogate model uses NSGA-II and SMPSO to find a solution aiming at an optimal latency in the network. Our findings show that the three TSN latency mechanisms can work together to reach a near-optimal latency, as suggested by both EO algorithms. The SMPSO had lower E2E latency and could compensate for some parameters by finding the optimal set.

In future works, we plan to develop new optimization models with actual TSN network data and compare the optimization results with the

optimized simulated results. It would lead to a better understanding of how accurate the simulated-based models are and how they deal with real data.

CRediT authorship contribution statement

Daniel Bezerra: Conceptualization, Methodology, Software, Validation, Formal analysis, Investigation, Writing – original draft, Writing – review & editing, Visualization. **Assis T. de Oliveira Filho:** Conceptualization, Methodology, Validation, Writing – review & editing. **Iago Richard Rodrigues:** Conceptualization, Methodology, Validation, Writing – review & editing. **Marrone Dantas:** Writing – review & editing. **Gibson Barbosa:** Writing – review & editing. **Ricardo Souza:** Supervision. **Judith Kelner:** Funding acquisition. **Djamel Sadok:** Writing – review & editing, Project administration.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

No data was used for the research described in the article.

References

[1] Ahmed Nasrallah, Akhilesh S. Thyagaturu, Ziyad Alharbi, Cuixiang Wang, Xing Shao, Martin Reisslein, Hesham ElBakoury, Ultra-low latency (ULL) networks: The IEEE TSN and IETF DetNet standards and related 5G ULL research, IEEE Commun. Surv. Tutor. 21 (1) (2019) 88–145.

[2] Silviu S. Craciunas, Ramon Serna Oliver, Martin Chmelík, Wilfried Steiner, Scheduling real-time communication in IEEE 802.1Qbv time sensitive networks, in: Proceedings of the 24th International Conference on Real-Time Networks and Systems, RTNS '16, Association for Computing Machinery, New York, NY, USA, 2016, pp. 183–192.

[3] IEEE-802.1 time-sensitive networking task group, 2022, https://www.ieee802.org/1/pages/tsn.html. Accessed: 2022-01-18.

[4] Mowei Wang, Yong Cui, Xin Wang, Shihan Xiao, Junchen Jiang, Machine learning for networking: Workflow, advances and opportunities, IEEE Netw. 32 (2) (2018) 92–99.

[5] Gaetano F. Anastasi, Massimo Coppola, Patrizio Dazzi, Marco Distefano, QoS guarantees for network bandwidth in private clouds, Procedia Comput. Sci. 97 (2016) 4–13, 2nd International Conference on Cloud Forward: From Distributed to Complete Computing.

[6] 802.1Qbu - frame preemption, 2022, https://www.ieee802.org/1/pages/802.1bu.html. Accessed: 2022-01-20.

[7] 802.1Qav - forwarding and queuing enhancements for time-sensitive streams, 2022, https://www.ieee802.org/1/pages/802.1av.html. Accessed: 2022-01-18.

[8] 802.1Qbv - enhancements for scheduled traffic, 2022, https://www.ieee802.org/1/pages/802.1bv.html. Accessed: 2022-01-20.

[9] IEEE standard for local and metropolitan area networks— bridges and bridged networks - amendment 24: Path control and reservation, in: IEEE Std 802.1Qca-2015 (Amendment to IEEE Std 802.1Q-2014 as Amended By IEEE Std 802.1Qcd-2015 and IEEE Std 802.1Q-2014/Cor 1-2015), 2016, pp. 1–120.

[10] IEEE Standard for Local and Metropolitan Area Networks - Timing and Synchronization for Time-Sensitive Applications in Bridged Local Area Networks, IEEE Std 802.1AS-2011, 2011, pp. 1–292.

[11] Dynamic latency guarantee, 2022, https://tools.ietf.org/id/draft-liu-detnet-dynamic-latency-guarantee-01.html. Accessed: 2022-01-18.

[12] Ieee standard for local and metropolitan area networks–bridges and bridged networks - amendment 34:asynchronous traffic shaping, in: IEEE Std 802.1Qcr-2020 (Amendment To IEEE Std 802.1Q-2018 As Amended By IEEE Std 802.1Qcp-2018, IEEE Std 802.1Qcc-2018, IEEE Std 802.1Qcy-2019, and IEEE Std 802.1Qcx-2020), 2020, pp. 1–151.

[13] Zifan Zhou, Juho Lee, Michael Stübert Berger, Sungkwon Park, Ying Yan, Simulating TSN traffic scheduling and shaping for future automotive ethernet, J. Commun. Netw. 23 (1) (2021) 53–62.

[14] Ahmed Nasrallah, Akhilesh S. Thyagaturu, Ziyad Alharbi, Cuixiang Wang, Xing Shao, Martin Reisslein, Hesham Elbakoury, Performance comparison of IEEE 802.1 TSN time aware shaper (TAS) and asynchronous traffic shaper (ATS), IEEE Access 7 (2019) 44165–44181.

[15] IEEE Standard for Local and metropolitan area networks–Bridges and Bridged Networks, IEEE Std 802.1Q-2014 (Revision of IEEE Std 802.1Q-2011), 2014, pp. 1–1832.

[16] Hyung-Taek Lim, Daniel Herrscher, Martin Johannes Waltl, Firas Chaari, Performance analysis of the IEEE 802.1 ethernet audio/video bridging standard, in: Proceedings of the 5th International ICST Conference on Simulation Tools and Techniques, SIMUTOOLS '12, ICST (Institute for Computer Sciences, Social-Informatics and Telecommunications Engineering), Brussels, BEL, 2012, pp. 27–36.

[17] Luxi Zhao, Paul Pop, Zhong Zheng, Hugo Daigmorte, Marc Boyer, Latency analysis of multiple classes of AVB traffic in TSN with standard credit behavior using network calculus, IEEE Trans. Ind. Electron. 68 (10) (2021) 10291–10302.

[18] IEEE Standard for Local and Metropolitan Area Networks–Timing and Synchronization for Time-Sensitive Applications, IEEE Std 802.1AS-2020 (Revision of IEEE Std 802.1AS-2011), 2020, pp. 1–421.

[19] Lucia Lo Bello, Wilfried Steiner, A perspective on IEEE time-sensitive networking for industrial communication and automation systems, Proc. IEEE 107 (6) (2019) 1094–1120.

[20] Mohammad Ashjaei, Mikael Sjödin, Saad Mubeen, A novel frame preemption model in TSN networks, J. Syst. Archit. 116 (2021) 102037.

[21] Mubarak Adetunji Ojewale, Patrick Meumeu Yomsi, Borislav Nikolić, Worst-case traversal time analysis of TSN with multi-level preemption, J. Syst. Archit. 116 (2021) 102079.

[22] Hanphil Lee, Juho Lee, Chulsun Park, Sungkwon Park, Time-aware preemption to enhance the performance of audio/video bridging (AVB) in IEEE 802.1 TSN, in: 2016 First IEEE International Conference on Computer Communication and the Internet (ICCCI), 2016, pp. 80–84.

[23] Daniel Thiele, Rolf Ernst, Formal worst-case performance analysis of time-sensitive ethernet with frame preemption, in: 2016 IEEE 21st International Conference on Emerging Technologies and Factory Automation (ETFA), 2016, pp. 1–9.

[24] Lucia Lo Bello, Mohammad Ashjaei, Gaetano Patti, Moris Behnam, Schedulability analysis of time-sensitive networks with scheduled traffic and preemption support, J. Parallel Distrib. Comput. 144 (2020) 153–171.

[25] Jiayi Zhang, Lihao Chen, Tongtong Wang, Xinyuan Wang, Analysis of TSN for industrial automation based on network calculus, in: 2019 24th IEEE International Conference on Emerging Technologies and Factory Automation (ETFA), 2019, pp. 240–247.

[26] Dorin Maxim, Ye-Qiong Song, Delay analysis of AVB traffic in time-sensitive networks (TSN), in: Proceedings of the 25th International Conference on Real-Time Networks and Systems, RTNS '17, Association for Computing Machinery, New York, NY, USA, 2017, pp. 18–27.

[27] Feng He, Lin Zhao, Ershuai Li, Impact analysis of flow shaping in ethernet-AVB/TSN and AFDX from network calculus and simulation perspective, Sensors (Basel, Switzerland) 17 (2017).

[28] Jörn Migge, Josetxo Villanueva, Nicolas Navet, Marc Boyer, Insights on the performance and configuration of AVB and TSN in automotive ethernet networks, in: 9th European Congress on Embedded Real Time Software and Systems (ERTS 2018), Toulouse, France, 2018.

[29] Luca Sciullo, Sushmit Bhattacharjee, Matthias Kovatsch, Bringing deterministic industrial networking to the W3C web of things with TSN and OPC UA, in: Proceedings of the 10th International Conference on the Internet of Things, IoT '20, Association for Computing Machinery, New York, NY, USA, 2020.

[30] Voica Gavrilu, Luxi Zhao, Michael L. Raagaard, Paul Pop, AVB-aware routing and scheduling of time-triggered traffic for TSN, IEEE Access 6 (2018) 75229–75243.

[31] Silviu S. Craciunas, Ramon Serna Oliver, Martin Chmelík, Wilfried Steiner, Scheduling real-time communication in IEEE 802.1Qbv time sensitive networks, in: Proceedings of the 24th International Conference on Real-Time Networks and Systems, RTNS '16, Association for Computing Machinery, New York, NY, USA, 2016, pp. 183–192.

[32] Tieu Long Mai, Nicolas Navet, Jörn Migge, On the use of supervised machine learning for assessing schedulability: Application to ethernet TSN, in: Proceedings of the 27th International Conference on Real-Time Networks and Systems, RTNS '19, Association for Computing Machinery, New York, NY, USA, 2019, pp. 143–153.

[33] Tieu Long Mai, Nicolas Navet, Improvements to deep-learning-based feasibility prediction of switched ethernet network configurations, in: 29th International Conference on Real-Time Networks and Systems, RTNS '2021, Association for Computing Machinery, New York, NY, USA, 2021, pp. 89–99.

[34] Tieu Long Mai, Nicolas Navet, Jörn Migge, A hybrid machine learning and schedulability analysis method for the verification of TSN networks, in: 2019 15th IEEE International Workshop on Factory Communication Systems (WFCS), 2019, pp. 1–8.

[35] András Varga, Rudolf Hornig, An overview of the OMNeT++ simulation environment, in: Proceedings of the 1st International Conference on Simulation Tools and Techniques for Communications, Networks and Systems & Workshops, 2008, pp. 1–10.

[36] Jonathan Falk, David Hellmanns, Ben Carabelli, Naresh Nayak, Frank Dürr, Stephan Kehrer, Kurt Rothermel, NeSTiNg: Simulating IEEE time-sensitive networking (TSN) in OMNeT++, in: Proceedings of the 2019 International Conference on Networked Systems (NetSys), Garching b. München, Germany, 2019.

[37] P802.1AS-rev – timing and synchronization for time-sensitive applications, 2022, https://1.ieee802.org/tsn/802-1as-rev/. Accessed: 2022-01-20.

[38] 802.1Q - virtual LANs, 2022, https://www.ieee802.org/1/pages/802.1Q.html. Accessed: 2022-01-20.

[39] Moez Ali, PyCaret: An open source, low-code machine learning library in python, 2020, PyCaret version 1.0.

[40] Antonio Benítez-Hidalgo, Antonio J. Nebro, José García-Nieto, Izaskun Oregi, Javier Del Ser, Jmetalpy: A python framework for multi-objective optimization with metaheuristics, Swarm Evol. Comput. 51 (2019) 100598.